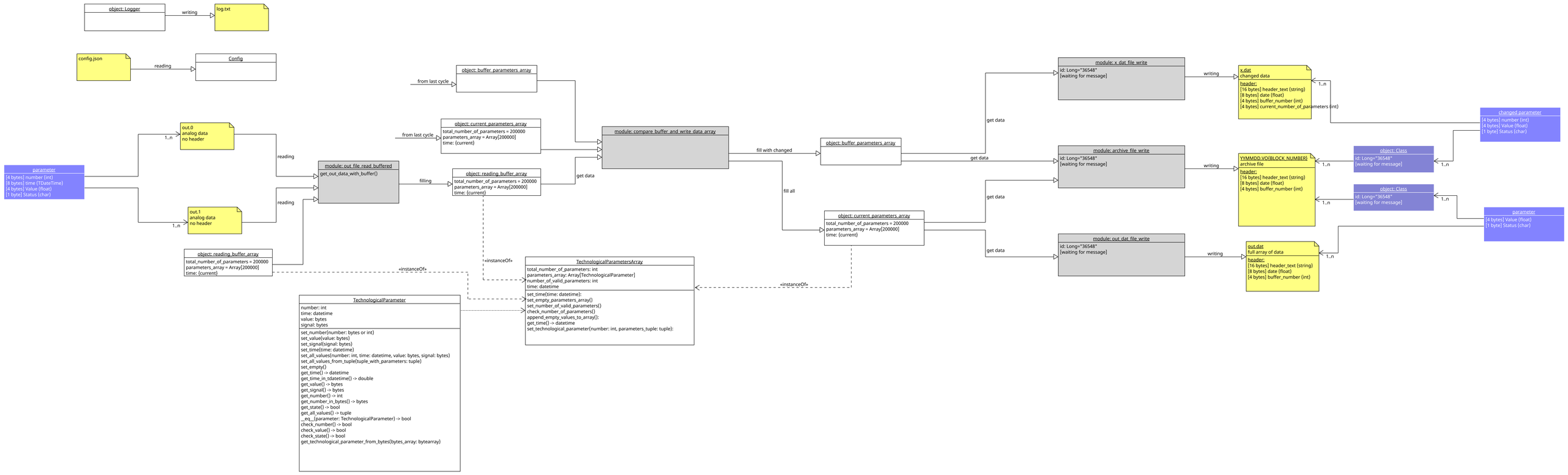
object: Logger
writing
log.txt
config.json
reading
Config
from last cycle
object: buffer_parameters_array
from last cycle
object: current_parameters_array
total_number_of_parameters = 200000
parameters_array = Array[200000]
time: {current}
module: compare_buffer_and_write_data_array
fill with changed
object: buffer_parameters_array
fill all
object: current_parameters_array
total_number_of_parameters = 200000
parameters_array = Array[200000]
time: {current}
get data
get data
get data
get data
module: x_dat_file_write
id: Long="36548"
[waiting for message]
writing
x.dat
changed data
header:
[16 bytes] header_text (string)
[8 bytes] date (float)
[4 bytes] buffer_number (int)
[4 bytes] current_number_of_parameters (int)
1..n
changed parameter
[4 bytes] number (int)
[4 bytes] Value (float)
[1 byte] Status (char)
module: archive_file_write
id: Long="36548"
[waiting for message]
writing
YYMMDD.VO[BLOCK_NUMBER]
archive file
header:
[16 bytes] header_text (string)
[8 bytes] date (float)
[4 bytes] buffer_number (int)
1..n
object: Class
id: Long="36548"
[waiting for message]
1..n
object: Class
id: Long="36548"
[waiting for message]
1..n
parameter
[4 bytes] Value (float)
[1 byte] Status (char)
module: out_dat_file_write
id: Long="36548"
[waiting for message]
writing
out.dat
full array of data
header:
[16 bytes] header_text (string)
[8 bytes] date (float)
[4 bytes] buffer_number (int)
1..n
parameter
[4 bytes] number (int)
[8 bytes] time (TDateTime)
[4 bytes] Value (float)
[1 byte] Status (char)
1..n
out.0
analog data
no header
1..n
out.1
analog data
no header
1..n
reading
reading
module: out_file_read_buffered
get_out_data_with_buffer()
filling
object: reading_buffer_array
total_number_of_parameters = 200000
parameters_array = Array[200000]
time: {current}
get data
object: reading_buffer_array
total_number_of_parameters = 200000
parameters_array = Array[200000]
time: {current}
«instanceOf»
«instanceOf»
«instanceOf»
TechnologicalParametersArray
total_number_of_parameters: int
parameters_array: Array[TechnologicalParameter]
number_of_valid_parameters: int
time: datetime
set_time(time: datetime):
set_empty_parameters_array()
set_number_of_valid_parameters()
check_number_of_parameters()
append_empty_values_to_array():
get_time() -> datetime
set_technological_parameter(number: int, parameters_tuple: tuple):
TechnologicalParameter
number: int
time: datetime
value: bytes
signal: bytes
set_number(number: bytes or int)
set_value(value: bytes)
set_signal(signal: bytes)
set_time(time: datetime)
set_all_values(number: int, time: datetime, value: bytes, signal: bytes)
set_all_values_from_tuple(tuple_with_parameters: tuple)
set_empty()
get_time() -> datetime
get_time_in_tdatetime() -> double
get_value() -> bytes
get_signal() -> bytes
get_number() -> int
get_number_in_bytes() -> bytes
get_state() -> bool
get_all_values() -> tuple
__eq__(parameter: TechnologicalParameter) -> bool
check_number() -> bool
check_value() -> bool
check_state() -> bool
get_technological_parameter_from_bytes(bytes_array: bytearray)